

Failure Analysis of Retrieval-Augmented Generation (RAG)

Problem Statement

Retrieval-Augmented Generation (RAG) improves the factual accuracy of Large Language Models (LLMs) by retrieving relevant context before generating answers. This project investigates how a baseline RAG pipeline performs on extractive question answering using the SQuAD v2 dataset and explores the impact of prompt engineering on answer quality.

Solution

A modular RAG pipeline was implemented using **Python**, **Sentence-Transformers**, **FAISS**, and the **Groq Llama 3.3 70B** model. The workflow consists of:

SQuAD v2 → Text Chunking → Embedding Generation → FAISS Index → Top-K Retrieval → Prompt Construction → LLM Answer Generation → Evaluation

The system evaluates performance using **Exact Match (EM)** and **Token F1**, allowing different retrieval parameters to be tested without changing the overall architecture.

Experiment

The first experiment focused on **prompt engineering** while keeping retrieval, embeddings, and evaluation unchanged.

The baseline prompt was replaced with a stricter prompt that instructed the model to:

- Answer only from the retrieved context.
- Return only the final answer.
- Avoid explanations or extra text.
- Respond with **"I don't know."** if the answer was unavailable.

Results

Configuration	EM	Token F1
Baseline Prompt	0.37	0.53
Improved Prompt	0.62	0.682

The revised prompt significantly improved both metrics. Many baseline errors occurred because the model produced correct but verbose answers, which were penalized by the Exact Match metric. Restricting the output to concise answers substantially increased performance without modifying retrieval or embeddings.

Conclusion

This project demonstrates that prompt engineering can significantly improve RAG performance even when the retrieval pipeline remains unchanged. The implementation provides a clean, reproducible baseline that can be extended with experiments involving chunk size, Top-K retrieval, embedding models, and retrieval failure analysis. During additional experimentation, the Groq free-tier API token quota was exhausted, limiting further runs with the same API key. However, the completed implementation and reported results are fully reproducible using a valid Groq API key.

